

Question : What does REST mean in the context of Web APIs?

Answer : REST is a style for designing APIs where resources are represented by URLs and HTTP methods are used to perform operations on them

Question : What is the difference between GET, POST, PUT, PATCH and DELETE?

Answer :

GET: Retrieve data (api/books)

POST: Create new data

PUT: Update the whole resource

PATCH: Update part of a resource

DELETE: Delete a resource

Question : When should an API return 200, 201, 204, 400 and 404?

Answer :

200 OK: The request succeeded and returns data

201 Created: A new resource was created

204 No Content: The request succeeded but there is no response body

400 Bad Request: The request data is invalid (in ProductsApi (Price<0))

404 Not Found: The requested resource does not exist (Like Get By Id Or Delete)

Question : What is the difference between route parameters and query parameters?

Answer :

Route parameters identify a specific resource (GET /api/products/5)

Query parameters are usually used for optional things like filtering, searching, or pagination
(GET /api/products?name=laptop)

Question : What is the role of a controller in ASP.NET Core Web API?

Answer :

The controller receives HTTP requests, calls the appropriate service, and returns the HTTP response. It should not contain the main business logic (Controller – IService –Service)

Question : Why should we use DTOs instead of exposing models directly?

Answer :

DTOs control the request and response shape and prevent exposing internal model details. For example, I used CreateProductRequest for creating products and ProductResponse for returning products

Question : Why should business logic not stay inside the controller?

Answer :

To keep the controller clean and make the business logic easier to test, reuse, and maintain

Question : What is Dependency Injection and why is it useful?

Answer :

Dependency Injection means providing a class with the dependencies it needs instead of creating them inside the class. It reduces coupling and makes the code easier to test

For example:

```
public ProductsController(IProductService productService)
```

Question : Where can validation happen in a Web API?

Answer :

Validation can happen in DTOs using Data Annotations, in the controller, or in the service for business rules

Question : What is Swagger/OpenAPI used for?

Answer :

Swagger provides interactive API documentation and allows us to test endpoints directly from the browser

Question : Why do we need Postman if Swagger already exists?

Answer :

Postman provides more advanced API testing, such as collections, environments, headers, authentication, and saved requests

Question : What makes an API response professional and predictable?

Answer :

It should have a consistent structure, meaningful HTTP status codes, clear data, and clear error messages

Question : How would you implement search and filtering in an API?

Answer :

GET /api/products?name=laptop&minPrice=1000

Then the service filters the data based on these parameters

Question : Why is pagination important in APIs?

Answer :

Pagination prevents returning a huge amount of data at once. It improves performance and reduces the response size (GET /api/products?page=2&pageSize=10)

Question : What is the difference between UI, controller, service and data storage?

Answer :

UI: Sends requests and displays results

Controller: Handles HTTP requests and responses

Service: Contains business logic

Data Storage: Stores and retrieves data

Question : What should reviewers see in your commit history?

Answer :

They should see clear and meaningful commits that show the development steps instead of one huge commit containing everything

Question : How do you prove that your API works without running it on the reviewer device?

Answer :

I can provide Swagger or Postman screenshots and a Postman collection showing successful and error responses

Question : How do you investigate an endpoint that returns 500?

Answer :

I check the exception details, application logs, debugger, and the controller/service code to find the source of the unexpected error

Question : Why should we not trust request data from users?

Answer :

Because users can send invalid or malicious data. We should always validate input before using it

Question : Why is in-memory storage not enough for real applications?

Answer :

Because the data is lost when the application stops or restarts. Real applications need persistent storage such as a database